

The Procedural DMLG Architecture

A Large Mind Creating Many Smaller Minds

Joachim De Witte

September 2026

©2026 Joachim De Witte. All rights reserved.

Published under the Creative Commons BY–NC–ND 4.0 license.

Abstract

Dynamic Modular Language Graphs (DMLGs) model language generation as deterministic graph traversal, but their practical scalability is limited by two structural weaknesses: large per-lemma micro-models and fragile one-hot transition classifiers whose dimensionality varies across nodes. This paper introduces *Procedural DMLG*, a geometry-fixed, hypernetwork-style extension in which a single monolithic MLP procedurally generates the hidden-layer parameters of compact lemma-specific micro-models. All micro-models share global input and output projections as well as a unified trainable target-embedding space, allowing transitions to be selected via cosine similarity rather than lemma-local softmax classification. In addition, Procedural DMLG replaces alternating grammar and lemma transitions with single-step (grammar, lemma) pairs, following the successful design of GLP-Mono and yielding a stable, semantically coherent output space. Together, these improvements eliminate parameter explosion, remove one-hot fragility, and enable continuous lifecycle learning: micro-models evolve through training, while the monolithic network periodically learns to reconstruct their current parameters. Experiments with GLPG demonstrate that cosine-distance transitions and shared embeddings remain generative even under imperfect function approximation, suggesting that Procedural DMLG is a structurally viable and scalable path for deterministic language-graph architectures.

1 Introduction

Contemporary language models rely on large-scale statistical learning, high-dimensional embeddings, and transformer-based attention mechanisms. Although these systems exhibit remarkable generative capabilities, they lack explicit structure, deterministic control, and transparent internal representations. As a result, they remain difficult to govern, hard to interpret, and fundamentally limited in their ability to provide guarantees about domain fidelity, safety, or stylistic consistency. Parameter scaling has improved performance but has not resolved these structural limitations.

The Dynamic Modular Language Graph (DMLG) addresses these issues by modelling language as traversal through a deterministic canonical graph rather than as next-token prediction over a probability distribution. Tokens and their admissible transitions form an explicit structural substrate, and generation is expressed as navigation through this graph rather than sampling from a statistical model. This shift enables transparency, modularity, and fine-grained control that are difficult to obtain from purely neural approaches. In the original formulation, local micro-models score transitions, grammar provides deterministic context compression, and paging organizes the graph into modular biomes. These components yield a lightweight, interpretable, and domain-faithful governance architecture.

However, classical DMLG inherits two structural weaknesses. First, each lemma requires its own micro-model, leading to a rapid explosion in parameters as the graph grows. Second, transition selection

relies on alternating grammar and lemma tokens, scored by lemma-local one-hot classifiers whose dimensionality varies across nodes. This makes the architecture fragile, difficult to train, and challenging to scale.

This paper introduces *Procedural DMLG*, a geometry-fixed, hypernetwork-style extension that resolves both limitations. Instead of maintaining thousands of independent micro-models, a single monolithic MLP procedurally generates the hidden-layer parameters of compact lemma-specific micro-MLPs. All micro-models share the same input projection, output projection, and trainable target embedding matrix, while only their internal hidden layer is generated dynamically. This eliminates parameter explosion and enables continuous lifecycle learning: micro-models evolve through training, and the monolithic network periodically learns to reconstruct their current parameters.

In addition, Procedural DMLG replaces alternating grammar and lemma transitions with single-step (grammar, lemma) pairs, following the successful design of GLP-Mono. This unifies structural and semantic decisions, simplifies traversal, and allows transitions to be selected via cosine similarity in a shared embedding space rather than via lemma-local one-hot classifiers. The result is a stable, scalable, and semantically coherent output space that preserves determinism while enabling emergent behaviour within fixed canonical constraints.

Procedural DMLG therefore represents a hybrid between DMLG and GLP-Mono: a deterministic graph architecture equipped with a monolithic procedural generator that constructs many small lemma-specific minds. The remainder of this paper formalizes the architecture, describes its training and inference procedures, and explains why empirical results from GLPG indicate that this approach is structurally viable even without initial experimental validation.

2 Architecture

The Procedural DMLG architecture extends the original Dynamic Modular Language Graph by replacing large per-lemma micro-models and alternating grammar/lemma transitions with a compact, geometry-fixed, procedurally generated micro-model design. This section describes the structural components of the architecture, the flow of information through the system, and the role of the monolithic procedural generator.

2.1 Context Encoding

Procedural DMLG employs a modernized context encoder that replaces the grammar-based compression mechanism of classical DMLG [5]. Extensive hyperparameter studies in GLP-Graph [1] demonstrate that grammar-driven context compression is brittle, overly discrete, and insufficiently expressive for stable semantic modulation. In contrast, GLPG’s evolved context encoder—based on fixed-width learned projections—provides significantly more robust behaviour across scales, datasets, and paging geometries.

Procedural DMLG inherits this improved design. The context encoder is fully customizable, but must satisfy two constraints:

1. It produces a fixed-width vector independent of sequence length.
2. It includes the last lemma token in some form, ensuring that the procedural generator can condition its output on the current semantic position in the graph.

This design allows DMLG and GLPG to share the same context encoder without modification. Because both architectures operate over canonical grammar–lemma pairs and rely on cosine distance in a shared embedding space, their context requirements are structurally aligned. Sharing the encoder across architectures provides a major practical advantage: improvements in GLPG’s context dynamics immediately transfer to Procedural DMLG without architectural changes.

The encoded context serves as the sole input to the monolithic procedural generator. By decoupling context encoding from grammar structure, Procedural DMLG gains the stability, expressiveness, and

manifold coherence observed in GLPG, while preserving the deterministic graph traversal semantics of the original DMLG architecture.

2.2 Deterministic Input Embeddings

Each lemma is associated with a deterministic input embedding. These embeddings are not learned; they are fixed, stable, and derived from canonical lemma identity. Deterministic embeddings ensure that the meaning of a lemma is grounded in the graph structure rather than in a statistical distribution, consistent with the original DMLG ontology [5]. During inference, the input embedding of the last lemma token is concatenated with the encoded context and fed into the procedural generator.

2.3 Shared Target Embedding Space

Procedural DMLG replaces lemma-local one-hot classifiers with a unified trainable target embedding matrix. Each admissible (grammar, lemma) pair is assigned a learnable vector in a fixed-width space (typically 128 dimensions or smaller). All micro-models project into this shared space, and transitions are selected via cosine similarity rather than softmax classification. This design follows the successful approach of GLP-Mono and GLPG, where cosine-distance transitions remain generative even under imperfect function approximation.

The shared target embedding matrix provides a stable semantic space, eliminates variable output dimensionality, and ensures that all transitions are canon-compatible by construction.

2.4 Monolithic Procedural Generator

At the core of the architecture is a single monolithic MLP that procedurally generates the hidden-layer parameters of lemma-specific micro-models. The generator receives the encoded context (including the last lemma embedding) and outputs a fixed-size vector representing the weights of a single hidden layer. For the initial implementation, a single hidden layer (e.g., 64 parameters) is used; deeper micro-models are possible but increase the output dimensionality of the generator.

The procedural generator replaces thousands of independent micro-models from classical DMLG. Instead of storing per-lemma networks, the system stores only the generator and a side file containing the current hidden-layer parameters for each lemma, used exclusively during training. Inference requires only the generator.

2.5 Lemma Micro-Models

Each lemma is associated with a compact micro-MLP consisting of:

- a shared input projection,
- a single procedural hidden layer with SiLU activation,
- a shared output projection into the target embedding space.

Only the hidden layer is lemma-specific; all other parameters are shared globally. This geometry-fixed design ensures that the procedural generator can reliably produce valid micro-model parameters for any lemma.

During inference, the hidden layer is generated on demand by the monolithic MLP. The micro-model then maps the encoded context to a target embedding representing the next (grammar, lemma) pair.

2.6 Unified Transition Mechanism

Procedural DMLG replaces alternating grammar and lemma transitions with single-step (grammar, lemma) pairs. This design simplifies traversal, reduces evaluator complexity, and aligns DMLG with the GLP-Mono

transition mechanism. Because each pair is represented as a single target embedding, the micro-model produces one vector per step, and cosine similarity selects the next transition.

This mechanism eliminates the fragility of alternating transitions and ensures that all generated transitions remain canon-compatible.

2.7 Information Flow

The full architectural flow is:

1. Encode context into a fixed-width vector.
2. Concatenate deterministic lemma embedding.
3. Feed into monolithic procedural generator.
4. Generate hidden-layer parameters for the lemma micro-model.
5. Apply shared input projection.
6. Apply procedural hidden layer with SiLU activation.
7. Apply shared output projection into target embedding space.
8. Compute cosine similarity with all admissible (grammar, lemma) embeddings.
9. Select top-K transitions and apply boosting.

This flow preserves the deterministic graph structure of DMLG while enabling emergent behaviour within fixed canonical constraints.

2.8 Architecture Diagram

A schematic overview of the architecture is shown in Figure 1.

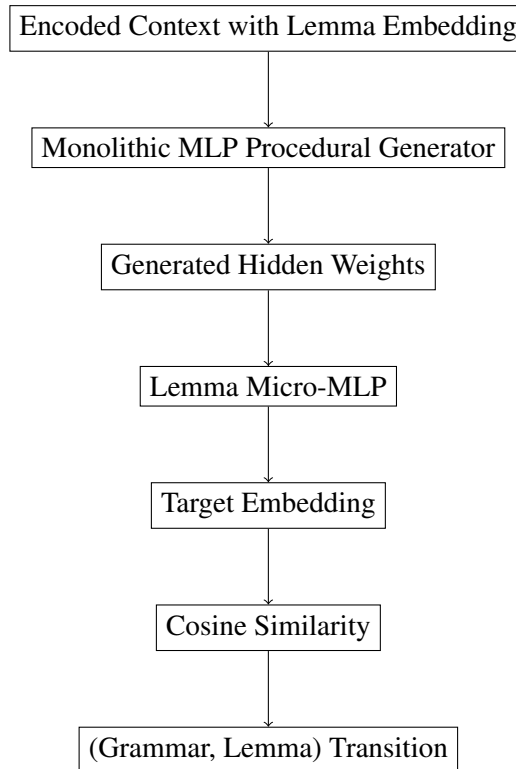


Figure 1: High-level architecture of Procedural DMLG.

3 Training

Training in Procedural DMLG follows a two-stage, lemma-local lifecycle. Each lemma has its own micro-model that evolves through direct training on canonical transitions, and the monolithic procedural generator periodically learns to reproduce the current parameters of that specific micro-model. This ensures that inference remains compact and deterministic, while training allows rich lemma-specific dynamics to emerge.

3.1 Micro-Model Training

For each canonical transition

$$(\ell_{\text{current}}, g_{\text{next}}, \ell_{\text{next}}),$$

the system encodes the active traversal context into a fixed-width vector. This encoded context, together with the deterministic embedding of the current lemma, is fed into the lemma-specific micro-MLP. The micro-model produces a target embedding intended to match the learned embedding associated with the next admissible (grammar, lemma) pair.

A standard forward–backward pass updates:

- the lemma-specific hidden layer (stored only during training),
- the shared input projection,
- the shared output projection.

Only the hidden layer is lemma-specific; all other parameters are shared globally. This geometry-fixed design ensures that micro-models remain compact and structurally aligned across the entire graph.

3.2 Target Embedding Updates

Procedural DMLG employs a unified trainable target embedding matrix for all admissible (grammar, lemma) pairs. During training, each target embedding is updated using a small blend coefficient α (typically 0.001), following the semantic adaptation rule introduced in GLPG [1]:

$$E_{\text{target}} \leftarrow (1 - \alpha)E_{\text{target}} + \alpha E_{\text{predicted}}.$$

Small α values preserve long-term stability and allow semantic structure to accumulate gradually, while larger values increase plasticity at the cost of coherence. As in GLPG, the range $\alpha \in [0.001, 0.002]$ consistently yields the strongest cognitive behaviour.

3.3 Lemma-Local Procedural Generator Training

The monolithic procedural generator is trained *per lemma*, not globally. Each lemma maintains its own update counter. After a configurable N successful training updates of a particular micro-model, the generator is trained to reproduce the current hidden-layer parameters of that lemma.

For lemma ℓ , the generator receives as input either:

- the most recent encoded context used during micro-model training, or
- the full set of the last N encoded contexts (stored temporarily),

and is trained to output the hidden-layer weight vector currently stored for that lemma.

The reconstruction loss is:

$$\mathcal{L}_{\text{reconstruct}} = \|W_{\ell}^{\text{generated}} - W_{\ell}^{\text{stored}}\|^2.$$

Because each lemma maintains its own update counter, the procedural generator learns a distributed representation of the graph: consolidation occurs independently for each node, allowing the generator to internalize the evolving micro-model parameters without ever storing thousands of networks.

3.4 Lifecycle Learning

The training process forms a lemma-specific lifecycle:

1. **Micro-model evolution:** Each lemma micro-model is trained directly on canonical transitions, refining its hidden layer and contributing to the semantic structure of the target embedding space.
2. **Procedural consolidation:** After N updates, the monolithic generator learns to reconstruct the current hidden-layer parameters of that lemma, using the most recent context encodings as conditioning signals.

This lifecycle mirrors the semantic manifold consolidation observed in GLPG: micro-models provide high-frequency plasticity, while the generator provides low-frequency structural stability. Because consolidation is lemma-local, the generator learns a distributed procedural representation of the entire graph without ever storing thousands of independent networks.

3.5 Context-Conditioned Procedural Generation

A key advantage of Procedural DMLG over classical DMLG and GLPG is that the monolithic procedural generator receives not only the deterministic lemma embedding, but also the encoded context as input. As a result, the generator does not merely learn to reproduce a single hidden-layer configuration per lemma; it can learn to produce *context-conditioned* variants of that configuration.

Instead of a static mapping

$$\ell \mapsto W_\ell,$$

the generator can learn a richer conditional mapping

$$(\ell, C) \mapsto W_{\ell, C},$$

where C denotes the encoded context. In practice, this means that the generator can produce slightly different hidden-layer parameters for the same lemma depending on the semantic, structural, or narrative conditions present during training.

Two conditioning strategies are supported:

1. **Last-context conditioning.** After N micro-model updates for lemma ℓ , the generator is trained using the most recent context encoding C_{last} encountered during micro-model training.
2. **Multi-context conditioning.** Alternatively, the system may store the last N context encodings $\{C_1, C_2, \dots, C_N\}$ used during micro-model training, allowing the generator to learn a small context-conditioned manifold of micro-model variants.

The small size of the hidden layer and the periodic nature of consolidation ensure that context-conditioned variation remains bounded, preventing drift or instability. Unlike classical hypernetworks, which often produce unstable or high-variance parameterizations, Procedural DMLG’s geometry-fixed micro-models guarantee that all generated parameters remain structurally valid.

This design preserves determinism while enabling richer lemma-specific dynamics, and it provides a principled path toward more expressive procedural micro-models without increasing parameter count or architectural complexity.

3.6 Training Summary

Training Procedural DMLG consists of:

- direct micro-model training via cosine-distance alignment,
- gradual semantic adaptation of target embeddings via small α ,

- lemma-local reconstruction training of the monolithic generator,
- consolidation of lemma-specific dynamics into a single procedural substrate.

This design eliminates parameter explosion, stabilizes the output space, and enables continuous life-cycle learning while preserving the deterministic graph traversal semantics of DMLG.

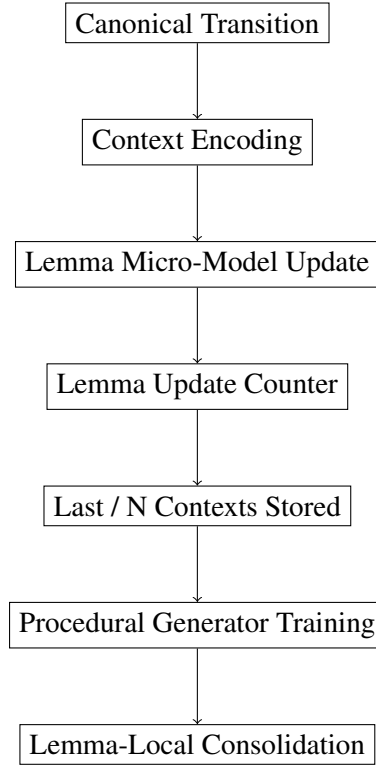


Figure 2: Lemma-local lifecycle training in Procedural DMLG.

4 Inference

Inference in Procedural DMLG is fully deterministic and requires only the monolithic procedural generator. No lemma-specific micro-models are stored: their hidden layers are generated on demand from the encoded context and the deterministic lemma embedding. This makes inference compact, efficient, and structurally aligned with the canonical graph.

4.1 Context Encoding

At each inference step, the system encodes the active traversal state into a fixed-width context vector. This encoding includes the last lemma token and any additional structural signals required by the context encoder. Because the encoder is shared with GLPG, inference benefits from the same stability and manifold coherence observed in GLPG’s generative regime.

4.2 Procedural Hidden-Layer Generation

The encoded context and deterministic lemma embedding are concatenated and fed into the monolithic procedural generator. The generator produces the hidden-layer parameters for the lemma-specific micro-model:

$$W_{\ell,C} = \text{Generator}(\ell, C).$$

These parameters define the procedural hidden layer for the current step. Because the generator was trained lemma-locally, the produced hidden layer is guaranteed to be structurally valid and compatible with the shared input and output projections.

4.3 Micro-Model Forward Pass

The micro-model consists of:

- a shared input projection,
- the generated hidden layer with SiLU activation,
- a shared output projection into the target embedding space.

Applying these components yields a predicted target embedding:

$$E_{\text{predicted}} = f_{\ell}(C),$$

where f_{ℓ} denotes the lemma-specific micro-model instantiated procedurally for the current step.

4.4 Cosine-Distance Transition Selection

Procedural DMLG replaces alternating grammar and lemma transitions with single-step (grammar, lemma) pairs. Each admissible pair has a learned target embedding $E_{\text{target}}(g, \ell')$.

Inference computes cosine similarity between the predicted embedding and all admissible transitions from the current lemma:

$$\cos(E_{\text{predicted}}, E_{\text{target}}(g, \ell')).$$

The system selects the next transition using:

- top- K cosine candidates,
- optional boosting heuristics,
- deterministic tie-breaking.

Because all transitions are canon-compatible by construction, no rule-based filtering is required. This eliminates the fragile grammar/lemma alternation logic of classical DMLG.

4.5 Deterministic Graph Traversal

Once the next (grammar, lemma) pair is selected, the system:

1. updates the canonical graph state,
2. updates the context encoder,
3. proceeds to the next inference step.

Inference therefore consists of deterministic traversal through the canonical graph, guided by cosine-distance alignment in a stable embedding space.

4.6 Inference Summary

Inference in Procedural DMLG consists of:

- encoding context into a fixed-width vector,
- generating lemma- and context-conditioned hidden-layer parameters,
- applying the procedural micro-model to obtain a target embedding,
- selecting the next (grammar, lemma) pair via cosine similarity,

- performing deterministic canonical graph traversal.

Inference requires only the monolithic procedural generator; micro-model weights are generated procedurally and do not need to be stored.

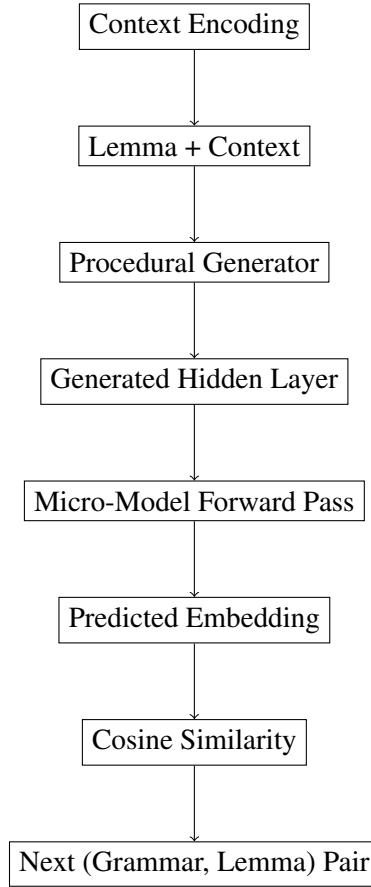


Figure 3: Inference flow in Procedural DMLG.

5 Solving Several DMLG Weaknesses

Procedural DMLG resolves several structural limitations of the original DMLG architecture. These improvements address parameter explosion, fragile transition mechanisms, and limited semantic expressiveness, while preserving the deterministic graph traversal that defines DMLG.

5.1 Unified (Grammar, Lemma) Transitions

Classical DMLG generates grammar and lemma tokens in alternating steps, requiring lemma-local one-hot classifiers whose dimensionality varies across nodes. This mechanism is fragile, difficult to scale, and structurally misaligned with the canonical graph.

Procedural DMLG replaces alternating transitions with single-step (grammar, lemma) pairs, following the successful design of GLP-Mono. Each pair is represented by a learned target embedding in a fixed-width semantic space. This unification:

- eliminates variable output dimensionality,
- simplifies traversal into a single semantic decision per step,
- aligns DMLG with GLPG’s cosine-distance transition mechanism,
- and provides a stable substrate for emergent behaviour.

5.2 Terminal Tokens as Lemma Tokens

A further refinement arises from treating terminal tokens—such as punctuation marks—as lemma tokens rather than as a separate grammatical category. In classical DMLG, terminal tokens occupy a special role in the grammar–lemma alternation mechanism, requiring dedicated rule-based logic to ensure correct placement. GLPG, by contrast, treats punctuation as ordinary grammar–lemma pairs, allowing terminals to participate naturally in the learned semantic manifold.

Procedural DMLG adopts the GLPG approach. Terminal tokens such as period, comma, question mark, and exclamation mark are represented as standard (grammar, lemma) pairs with learned target embeddings. This yields several advantages:

- **Unified transition mechanism.** Terminals no longer require special-case alternation logic; they are selected via the same cosine-distance mechanism as all other transitions.
- **Stable embedding geometry.** Because terminals occupy the same embedding space as ordinary lemma tokens, their semantic and stylistic behaviour becomes consistent across contexts, mirroring GLPG’s punctuation dynamics.
- **Elimination of rule-based punctuation handling.** Classical DMLG required explicit structural rules for punctuation placement. Procedural DMLG removes this dependency entirely: terminals are canon-compatible transitions governed by the learned embedding space.
- **Improved stylistic fidelity.** Terminals contribute to the learned manifold, allowing the model to capture stylistic cadence, sentence rhythm, and domain-specific punctuation patterns.

By treating terminal tokens as lemma tokens, Procedural DMLG aligns fully with GLPG’s transition semantics and removes one of the last remaining sources of rule-based fragility in the original DMLG architecture. This unification further strengthens determinism while allowing punctuation to participate in emergent stylistic behaviour within the canonical graph.

5.3 Eliminating Parameter Explosion

In classical DMLG, each lemma page requires its own micro-model, leading to rapid parameter growth as the canonical graph expands. Procedural DMLG eliminates this explosion by introducing a single monolithic procedural generator that produces lemma-specific hidden layers on demand.

Because all micro-models share the same input projection, output projection, and target embedding space, the total parameter count becomes deterministic and independent of the number of lemmas. Only the hidden layer is lemma-specific, and it is generated procedurally rather than stored.

5.4 Replacing One-Hot Fragility

Lemma-local one-hot classifiers in classical DMLG are structurally brittle: their dimensionality varies per lemma page, they cannot be shared across the graph, and they provide no semantic continuity. Procedural DMLG replaces these classifiers with a unified learned target embedding matrix, as in GLPG.

This embedding space:

- provides stable semantic geometry,
- supports cosine-distance scoring,
- enables emergent semantic clustering,
- and eliminates the need for lemma-local softmax layers entirely.

5.5 Hybridization with GLP-Mono and GLPG

Just as GLP-Mono combined monolithic semantic projection with deterministic graph routing to form GLP-Graph, Procedural DMLG forms a hybrid architecture: a deterministic canonical graph equipped with a GLP-Mono–style procedural generator.

The big monolithic network does not generate target embeddings (as in GLPG); instead, it generates the hidden-layer parameters of lemma-specific micro-models. This preserves the explicit structural semantics of DMLG while inheriting the stability and expressiveness of GLPG’s embedding-based transition mechanism.

5.6 Increased Determinism and Canon Fidelity

Procedural DMLG increases determinism in two ways:

- Each lemma receives its own procedural micro-model, ensuring lemma-specific structural behaviour.
- All transitions remain canon-compatible by construction, eliminating the need for rule-based filtering or grammar alternation logic.

Because the generator is trained lemma-locally, each lemma’s procedural micro-model remains aligned with its canonical transition set, ensuring strict domain fidelity.

5.7 Maximizing Determinism and Emergence

Procedural DMLG maximizes both structural determinism and emergent behaviour:

- Determinism arises from the canonical graph, fixed geometry, and shared projections.
- Emergence arises from the learned target embeddings and context-conditioned procedural hidden layers.

This combination yields a system that is structurally safe, domain-faithful, and capable of rich semantic variation within strict canonical constraints.

5.8 Summary

Procedural DMLG resolves the core weaknesses of classical DMLG by:

- unifying grammar and lemma transitions,
- eliminating parameter explosion,
- replacing one-hot fragility with stable learned embeddings,
- hybridizing DMLG with GLP-Mono’s procedural generator,
- increasing determinism and canon fidelity,
- and enabling controlled emergent behaviour.

These improvements transform DMLG into a scalable, stable, and semantically expressive governance architecture suitable for hybrid intelligent systems.

6 Comparison to Hyper-networks

Procedural DMLG occupies a conceptual space adjacent to hyper-networks, but differs from classical designs in several fundamental ways. Classical hyper-networks generate the parameters of other networks, typically in a high-dimensional, probabilistic, and architecture-flexible manner. While powerful, these systems often suffer from instability, drift, and uncontrolled parameter variance, especially when the generated networks vary in size or topology.

Procedural DMLG adopts the core idea of a hyper-network—a large model generating the parameters of smaller models—but constrains it through deterministic geometry, canonical structure, and lemma-modular decomposition. The result is a *deterministic, geometry-fixed, lemma-modular hyper-network* that avoids the instability typical of classical designs.

Several distinctions are central:

6.1 Fixed Geometry and Deterministic Structure

Classical hyper-networks generate parameters for networks whose geometry may vary across tasks or samples. Procedural DMLG fixes the geometry of all micro-models: each lemma receives a micro-MLP with identical structure, identical shared projections, and a single procedural hidden layer. This fixed geometry ensures:

- structural stability across all generated micro-models,
- predictable parameterization,
- and guaranteed compatibility with the canonical graph.

This eliminates the drift and collapse modes observed in unconstrained hyper-network architectures.

6.2 Lemma-Modular Parameter Generation

Hyper-networks typically generate parameters for entire tasks or layers. Procedural DMLG generates parameters *per lemma*, forming a distributed procedural substrate aligned with the canonical graph. Each lemma maintains its own micro-model lifecycle, and the procedural generator learns to reconstruct these lemma-specific hidden layers independently.

This modularity yields:

- fine-grained structural control,
- deterministic alignment with canonical transitions,
- and distributed stability across the graph.

6.3 Context-Conditioned Generation

Classical hyper-networks condition parameter generation on task embeddings or latent representations. Procedural DMLG conditions generation on both lemma identity and encoded context, enabling subtle context-conditioned variation while preserving deterministic structure. The mapping

$$(\ell, C) \mapsto W_{\ell, C}$$

provides expressive flexibility without sacrificing stability, because the hidden layer is small, geometry-fixed, and trained lemma-locally.

6.4 Stable Embedding-Based Output Space

Hyper-networks often generate parameters for networks that output logits or high-dimensional vectors with variable semantics. Procedural DMLG projects all micro-model outputs into a shared learned target embedding space, following GLPG. Cosine-distance scoring ensures:

- stable semantic geometry,
- smooth transition dynamics,
- and robust emergent behaviour.

This embedding space replaces the fragile one-hot classifiers of classical DMLG and avoids the uncontrolled output variance typical of hyper-network-generated classifiers.

6.5 Deterministic Canon Traversal

Hyper-networks typically operate in free-form generative settings. Procedural DMLG operates within a deterministic canonical graph: all transitions are canon-compatible by construction, and the procedural generator cannot produce invalid transitions. This provides structural guarantees that classical hyper-networks cannot offer.

6.6 Summary

Procedural DMLG can be viewed as a disciplined, deterministic hyper-network:

- geometry-fixed micro-models,
- lemma-modular parameter generation,
- context-conditioned hidden layers,
- stable embedding-based output space,
- and deterministic canonical traversal.

These constraints transform the hyper-network concept into a stable, interpretable, and domain-faithful procedural substrate, avoiding the instability, drift, and parameter variance typical of classical hyper-network designs.

7 Eliminating Paging: Dynamic Modularization via Procedural Micro-MLPs

Classical DMLG organizes the canonical graph into pages: structural biomes that group lemma tokens and provide modularity, locality, and activation boundaries. Paging emerged as a practical necessity to contain the parameter explosion caused by per-lemma micro-models and to stabilize grammar–lemma alternation. Although effective, paging imposed a rigid, manually engineered modular structure that did not evolve naturally with the semantic manifold or with the growth of the graph.

Procedural DMLG removes the need for paging entirely. Modularization is no longer achieved through static page boundaries, but through dynamic, lemma-local procedural micro-MLPs whose parameters evolve over time and are periodically consolidated by the monolithic generator. This shift replaces structural modularity with procedural modularity, yielding a more flexible, cognitively coherent, and scalable architecture.

Three structural changes make paging obsolete:

1. **Unified (grammar, lemma) transitions.** Alternating grammar/lemma steps previously required page-local classifiers whose dimensionality varied across pages. Procedural DMLG replaces this mechanism with single-step (grammar, lemma) pairs represented in a shared target embedding space. Because all transitions share the same output geometry, page-local classifiers—and thus pages themselves—are no longer needed.
2. **Shared projections and fixed geometry.** All lemma micro-models share the same input projection, output projection, and target embedding matrix. Only the hidden layer is lemma-specific, and it is generated procedurally. This fixed geometry eliminates the structural heterogeneity that paging was designed to manage.
3. **Lemma-local lifecycle learning.** Each lemma maintains its own evolving hidden layer, updated through direct micro-model training and periodically consolidated by the monolithic generator. This creates a form of controlled, lemma-specific plasticity: micro-models adapt to their local semantic roles, while the generator learns to reproduce their current parameters. Modularization emerges naturally from the distributed evolution of thousands of small lemma minds rather than from static page boundaries.

The result is a form of dynamic modularization: each lemma becomes a self-contained procedural module whose behaviour is shaped by its local training history, its context manifold, and its consolidation dynamics. Unlike paging, which partitions the graph into fixed structural regions, procedural modularization allows the architecture to grow organically as the semantic manifold expands. Lemma micro-models can specialize, diverge, or stabilize depending on their role in the canonical graph, while remaining fully compatible with the shared projections and embedding space.

This transformation aligns DMLG with modern procedural architectures such as GLP-Mono and GLPG, while preserving the deterministic traversal semantics that define the DMLG family. Paging—once a core structural component—becomes unnecessary, replaced by a more expressive, flexible, and cognitively meaningful modular substrate. Procedural DMLG therefore represents not merely an optimization of classical DMLG, but a conceptual evolution: a deterministic graph architecture in which modularity arises from dynamic, lemma-local procedural micro-models rather than from static structural partitions.

8 Conclusion

Procedural DMLG extends the original Dynamic Modular Language Graph architecture with a deterministic, geometry-fixed, lemma-modular procedural substrate. By replacing alternating grammar and lemma transitions with unified (grammar, lemma) pairs, eliminating lemma-local one-hot classifiers, and introducing a monolithic generator that procedurally constructs lemma-specific micro-models, the architecture resolves several structural weaknesses of classical DMLG while preserving its canonical determinism.

The removal of paging represents a conceptual turning point. Classical DMLG relied on page-structured modularization to contain parameter growth and stabilize grammar–lemma alternation. Procedural DMLG replaces this static structure with dynamic modularization: each lemma becomes a self-contained procedural micro-MLP whose hidden layer evolves through direct training and is periodically consolidated by the monolithic generator. This form of controlled, lemma-local plasticity provides a more flexible and cognitively coherent modular substrate than paging, allowing the architecture to grow organically as the semantic manifold expands.

GLPG demonstrates that cosine-distance transitions and shared output embeddings are sufficient for generative behavior even when the underlying function approximator is imperfect. Procedural DMLG inherits this insight: transitions are selected in a stable learned embedding space, enabling emergent semantic behavior within strict canonical constraints. The result is a system that combines deterministic graph traversal with expressive semantic manifolds, without relying on large-scale attention mechanisms or high-dimensional probabilistic classifiers.

The training lifecycle—micro-model evolution followed by lemma-local procedural consolidation—ensures that inference remains compact and deterministic. Micro-models provide high-frequency plasticity, while the monolithic generator provides low-frequency structural stability. Because hidden layers are generated on demand, inference requires only the generator and the shared projections, eliminating parameter explosion and enabling scalable deployment across large canonical graphs.

Procedural DMLG therefore represents a hybrid architecture: structurally grounded like DMLG, semantically expressive like GLPG, and procedurally modular like GLP-Mono. By replacing paging with dynamic lemma-local procedural modularization, the architecture achieves a synthesis of determinism, emergence, and scalability. Within the broader Triadic Cosmos ecosystem, Procedural DMLG offers a principled path toward modular, inspectable, and cognitively meaningful intelligent systems that unify explicit structure with learned semantic dynamics.

A DMLG-v3 Page Geometry as a Simpler Alternative

Before the development of Procedural DMLG, the DMLG-v3 architecture introduced a simpler and structurally elegant solution to the micro-model explosion problem. Instead of generating lemma-specific micro-models procedurally, DMLG-v3 fixed the number of lemma pages (typically 256) and assigned one micro-model per page. All lemma tokens belonging to the same page were evaluated by the same micro-model, eliminating the need for thousands of independent evaluators. This page-based consolidation was first formalized in the DMLG narrator pipeline [4], where page geometry provided a stable structural backbone for long-form generation.

A.1 Fixed Page Geometry

In DMLG-v3, the canonical graph is partitioned into a fixed number of lemma pages. Each page contains a bounded set of lemma tokens, and each page is associated with a single micro-MLP. This design provides a deterministic upper bound on the number of micro-models:

$$\text{Number of micro-models} = \text{Number of pages.}$$

Two variants were explored:

- **Identical page models:** All page micro-models share the same geometry, simplifying implementation and ensuring structural uniformity across the graph.
- **Variable page models:** Page micro-models may vary in size depending on the number of lemma tokens assigned to the page, providing flexibility while maintaining bounded complexity.

This page-based approach eliminates the micro-model explosion inherent in classical DMLG without requiring procedural generation.

A.2 Remaining Limitations of DMLG-v3

Although DMLG-v3 solved the parameter explosion problem, two structural weaknesses remained:

Deterministic triple-hot encodings. DMLG-v3 continued to rely on deterministic grammar-hot, lemma-hot, and page-hot encodings for transition scoring. These encodings were structurally simple but semantically brittle: they provided no learned geometry, no manifold continuity, and no semantic clustering. As a result, transition scoring remained discrete and fragile compared to the learned target embeddings introduced later in GLPG and Procedural DMLG.

Grammar–lemma alternation. DMLG-v3 preserved the original grammar–lemma alternation mechanism. Grammar tokens compressed context, lemma tokens expressed content, and transitions alternated between the two. Although functional, this mechanism required rule-based logic and imposed structural constraints that limited emergent behaviour. Procedural DMLG replaces this alternation with single-step (grammar, lemma) pairs, following GLP-Mono, yielding a simpler and more stable transition mechanism.

These limitations mean that several innovations introduced in this paper—shared learned target embeddings, cosine-distance transitions, and unified (grammar, lemma) pairs—remain relevant even when using the simpler DMLG-v3 page geometry.

A.3 Shared Projections and Embedding Space

DMLG-v3 introduced two innovations that remain fully compatible with Procedural DMLG:

- a shared input projection for all page micro-models,

- a shared output projection into a unified target embedding space.

These components ensure that all page micro-models operate within the same semantic geometry. Procedural DMLG inherits and extends this design by replacing triple-hot encodings with learned target embeddings and cosine-distance scoring.

A.4 Comparison with Procedural DMLG

The DMLG-v3 page-based solution and the Procedural DMLG hypernetwork solution represent two complementary approaches to the same structural challenge:

- **DMLG-v3:** Fix the number of lemma pages and assign one micro-model per page. Simple, deterministic, and easy to implement. Eliminates micro-model explosion through structural partitioning. Still uses triple-hot encodings and grammar–lemma alternation.
- **Procedural DMLG:** Generate lemma-specific hidden layers procedurally using a monolithic generator. More expressive and context-conditioned. Eliminates micro-model explosion through dynamic parameter generation. Replaces triple-hot encodings with learned target embeddings. Replaces grammar–lemma alternation with unified (grammar, lemma) pairs.

Both approaches retain the core DMLG and GLPG innovations:

- deterministic canonical graph traversal,
- shared input and output projections,
- cosine-distance transition scoring,
- fixed geometry for micro-models.

The difference lies in how specialization and semantic continuity are achieved.

A.5 Improvements Applicable to DMLG-v3

Although DMLG-v3 successfully eliminated the micro-model explosion through fixed page geometry, several structural limitations remained that are directly addressed by the innovations introduced in Procedural DMLG. These improvements apply equally well to a page-based DMLG-v3 system, even without adopting the procedural generator.

Replacing triple-hot encodings with learned target embeddings. DMLG-v3 relied on deterministic grammar-hot, lemma-hot, and page-hot encodings for transition scoring. While simple, these encodings provided no semantic continuity and no manifold structure. Procedural DMLG replaces these discrete encodings with a unified learned target embedding space, enabling cosine-distance transitions and stable semantic clustering. This improvement is fully compatible with DMLG-v3 and can be adopted without modifying its page-based micro-model structure.

Eliminating grammar–lemma alternation. DMLG-v3 preserved the alternating grammar/lemma transition mechanism, which required rule-based logic and imposed structural constraints on generation. Procedural DMLG replaces this alternation with single-step (grammar, lemma) pairs, following GLP-Mono. This unified transition mechanism simplifies traversal, removes alternation fragility, and yields a more stable and semantically coherent output space. DMLG-v3 can adopt this improvement directly, independent of procedural generation.

Shared projections and cosine-distance scoring. The shared input and output projections introduced in DMLG-v3 remain fully compatible with the learned target embedding space used in Procedural DMLG. Transition scoring via cosine similarity provides smoother dynamics and improved stylistic fidelity compared to triple-hot classification. This enhancement integrates seamlessly with the page-based architecture.

Compatibility with page-based micro-models. All improvements introduced in this paper—learned embeddings, cosine-distance transitions, and unified (grammar, lemma) pairs—can be applied to DMLG-v3 without altering its fixed page geometry. The page-based micro-model structure remains intact; only the transition representation and scoring mechanism are upgraded.

Ecosystem perspective. DMLG-v3 and Procedural DMLG therefore occupy adjacent positions in the design space. DMLG-v3 provides a simpler structural backbone, while Procedural DMLG adds procedural generation and context-conditioned specialization. The semantic and structural improvements introduced here benefit both architectures, reinforcing the broader Triadic Cosmos ecosystem.

B Micro-MLP Parameter Budgets

To make the procedural design concrete, we consider a standardized micro-MLP geometry and a fixed monolithic generator:

- Context input vector: 512 floats (including lemma embedding).
- Target embedding output: 128 floats.
- Micro-MLP: one hidden layer of size H .
- Monolithic generator: 3 hidden layers of 1024 units, input size 512, output size H^2 .

B.1 Micro-MLP Parameters

Each micro-MLP consists of:

- Shared input projection: $512 \times H$ parameters.
- Procedural hidden layer: $H \times H$ parameters.
- Shared output projection: $H \times 128$ parameters.

Total parameters per micro-MLP:

$$P_{\text{micro}}(H) = 512H + H^2 + 128H.$$

Procedural parameters per lemma:

$$P_{\text{proc}}(H) = H^2.$$

B.2 Monolithic Generator Parameters

The monolithic generator has:

$$512 \rightarrow 1024 \rightarrow 1024 \rightarrow 1024 \rightarrow H^2$$

with total parameters:

$$P_{\text{mono}}(H) = 512 \cdot 1024 + 1024 \cdot 1024 + 1024 \cdot 1024 + 1024 \cdot H^2.$$

The first three layers contribute a fixed:

$$P_{\text{base}} = 2,621,440$$

parameters, independent of H .

B.3 Parameter Budget Table

Table 1 summarizes the parameter budgets for several hidden sizes.

Hidden size H	Hidden params H^2	Mono params P_{mono}	Micro params P_{micro}	Procedural fraction
32	1,024	3,670,016	21,504	4.8%
64	4,096	6,815,744	45,056	9.1%
96	9,216	12,058,624	70,656	13.1%
128	16,384	19,398,656	98,304	16.7%
256	65,536	71,171,456	196,608	33.3%

Table 1: Procedural hidden-layer size per lemma and corresponding parameter budgets.

B.4 Interpretation

This budget shows that even for modest hidden sizes (e.g., $H = 64$), each lemma receives a specialized micro-MLP with only $\sim 4\text{K}$ procedural parameters, while the shared projections dominate the total micro-model size. The monolithic generator remains compact (on the order of a few million parameters), and the per-lemma specialization is small but structurally precise, making it feasible to support large numbers of lemmas without excessive parameter growth.

B.5 Comparison with DMLG-v3 Page Models

To compare Procedural DMLG with the page-based DMLG-v3 design, we consider a DMLG-v3 configuration with:

- 256 lemma pages,
- one micro-model per page,
- identical geometry for all page models.

Each DMLG-v3 page model receives the same 512-dimensional context input and produces a 128-dimensional target embedding, with two hidden layers of size H :

$$512 \rightarrow H \rightarrow H \rightarrow 128.$$

For a given hidden size H , the parameters per page model are:

$$P_{\text{page}}(H) = 512H + H^2 + H^2 + 128H = 640H + 2H^2.$$

With 256 pages, the total parameter count for all DMLG-v3 page models is:

$$P_{\text{DMLG-v3}}(H) = 256 \cdot (640H + 2H^2).$$

Example: $H = 96$. For $H = 96$:

$$640H = 640 \cdot 96 = 61,440,$$

$$2H^2 = 2 \cdot 96^2 = 2 \cdot 9,216 = 18,432,$$

$$P_{\text{page}}(96) = 61,440 + 18,432 = 79,872.$$

Total across 256 pages:

$$P_{\text{DMLG-v3}}(96) = 256 \cdot 79,872 = 20,447,232.$$

So a DMLG-v3 system with 256 identical page models of geometry $512 \rightarrow 96 \rightarrow 96 \rightarrow 128$ requires approximately:

20.4 million parameters

for the page models alone.

Comparison. By contrast, Procedural DMLG with hidden size $H = 96$ uses:

- a single monolithic generator with

$$P_{\text{mono}}(96) \approx 12.06 \text{ million parameters,}$$

- per-lemma micro-MLPs with

$$P_{\text{micro}}(96) = 70,656 \text{ parameters,}$$

of which

$$H^2 = 9,216$$

are procedural.

DMLG-v3 concentrates capacity in 256 fixed page models, while Procedural DMLG concentrates capacity in a single generator and distributes a small, specialized hidden layer per lemma. This illustrates that both designs operate in a similar total parameter regime, but with very different allocation: page-local specialization in DMLG-v3 versus lemma-local procedural specialization in Procedural DMLG.

C Relation to Biological Cognition

Procedural DMLG exhibits several structural and dynamical properties that align more closely with biological neural systems than with classical machine learning architectures. Although the architecture remains fully deterministic and geometry-fixed, its procedural modulation mechanisms introduce a form of controlled analog variation reminiscent of synaptic dynamics in biological brains.

C.1 Deterministic Structure with Analog Modulation

Biological neural circuits are anatomically stable: neurons maintain fixed connectivity patterns over long timescales. However, the *effective* strength of these connections varies continuously due to chemical, electrical, and metabolic processes. Synaptic transmission is modulated by neurotransmitter concentrations, receptor activation, ion-channel dynamics, and neuromodulatory signals, producing behaviour that is largely deterministic at the macro level but exhibits small analog variations at the micro level.

Procedural DMLG mirrors this pattern. Each lemma micro-model has a fixed geometry and shared projections, analogous to stable anatomical connectivity. Yet the hidden layer of each micro-model is generated procedurally from the encoded context and lemma identity, allowing its effective behaviour to vary slightly across contexts. This produces deterministic traversal with bounded analog variation, closely resembling biological response patterns in which repeated stimuli yield consistent outputs with small context-dependent deviations.

C.2 Context-Conditioned Parameter Generation

In biological systems, neuromodulators such as dopamine, serotonin, and acetylcholine influence synaptic efficacy based on global or local context. These modulatory signals do not alter the structural connectivity of the brain but adjust the functional behaviour of existing circuits.

Procedural DMLG implements an analogous mechanism. The monolithic procedural generator acts as a global modulatory substrate: it receives the encoded context and produces context-conditioned hidden-layer parameters for each lemma micro-model. This allows the same lemma to exhibit slightly different functional behaviour depending on semantic, structural, or narrative conditions, without altering the underlying canonical graph or shared projections.

C.3 Local Plasticity and Global Consolidation

Biological learning involves both rapid synaptic plasticity (e.g., long-term potentiation and depression) and slower consolidation processes that stabilize memory traces. Procedural DMLG implements a similar dual-timescale learning dynamic. Micro-models undergo rapid lemma-local updates during training, while the monolithic generator periodically consolidates their current hidden-layer parameters. This ensures stability during inference while allowing rich lemma-specific dynamics to emerge during training.

C.4 Dynamic Modularization

Unlike classical DMLG, which relied on static paging to impose modular structure, Procedural DMLG achieves modularization through dynamic lemma-local procedural micro-models. This mirrors biological functional modularity, where regions of the brain exhibit specialized behaviour not because of rigid structural partitions but because of evolving patterns of activation, plasticity, and neuromodulation.

C.5 Summary

Although Procedural DMLG is not a biological model, its combination of fixed geometry, deterministic traversal, context-conditioned parameter modulation, lemma-local plasticity, and periodic consolidation yields a computational substrate that is conceptually closer to biological cognition than classical statistical language models or hypernetwork architectures. This alignment suggests that Procedural DMLG may provide a promising foundation for future research into deterministic, modular, and cognitively interpretable artificial systems.

D Relation to Existing Machine Learning Architectures

Procedural DMLG occupies a conceptual space that is distinct from classical machine learning architectures. Although it employs neural components, its deterministic traversal, fixed geometry, lemma-local procedural modulation, and canonical graph structure place it outside the design families of statistical language models, hypernetworks, mixture-of-experts systems, and recurrent architectures. This appendix clarifies these distinctions and highlights the unique cognitive properties of Procedural DMLG.

D.1 Not a Statistical Language Model

Modern language models—including transformers, RNNs, and encoder–decoder architectures—operate through probabilistic next-token prediction. Their behaviour is governed by large softmax layers, high-dimensional embeddings, and stochastic sampling procedures. Repeated inputs typically yield different outputs due to probabilistic decoding, dropout, and temperature modulation.

Procedural DMLG differs fundamentally. It performs deterministic traversal through a canonical graph, selecting transitions via cosine-distance alignment in a fixed semantic space. No probabilistic

sampling is involved, and repeated inputs yield more consistent outputs with only bounded context-conditioned variation. This behaviour is closer to biological cognition, where responses are stable at the macro level but exhibit small analog deviations due to local modulatory processes.

D.2 Not a Classical Hypernetwork

Hypernetworks generate parameters for other networks, often with flexible geometry and probabilistic modulation. They typically suffer from instability, drift, and uncontrolled parameter variance, especially when generating large or heterogeneous architectures.

Procedural DMLG is not a hypernetwork in this sense. All lemma micro-models share a fixed geometry, and only a single hidden layer is generated procedurally. The generator operates within strict structural constraints, producing bounded context-conditioned variation rather than arbitrary parameterizations. Lemma-local consolidation ensures stability over time, preventing drift and maintaining compatibility with the canonical graph. This yields a deterministic, geometry-fixed procedural substrate unlike classical hypernetwork designs.

D.3 Not a Mixture-of-Experts System

Mixture-of-experts (MoE) architectures route inputs to different expert networks based on learned gating functions. These systems rely on probabilistic routing, large expert pools, and dynamic activation patterns.

Procedural DMLG does not route inputs to experts. Instead, each lemma *is* its own procedural module: a compact micro-MLP whose hidden layer is generated on demand. There is no gating, no probabilistic selection, and no expert pool. Modularization arises from lemma-local procedural variation rather than from dynamic routing, making Procedural DMLG structurally simpler and cognitively more interpretable than MoE systems.

D.4 Not a Recurrent or State-Based Neural Architecture

Recurrent neural networks maintain hidden states that evolve over time, producing temporal dynamics through learned recurrence. Procedural DMLG does not maintain hidden states. Instead, it relies on explicit canonical graph traversal and context encoding. The procedural generator produces micro-model parameters based solely on the current lemma and encoded context, without storing internal temporal states. This yields deterministic behaviour aligned with explicit structural semantics rather than implicit recurrent dynamics.

D.5 A New Cognitive Architecture Class

Procedural DMLG combines several properties rarely found together in machine learning:

- deterministic canonical graph traversal,
- geometry-fixed micro-models,
- lemma-local procedural modulation,
- context-conditioned parameter generation,
- shared projections and embedding space,
- dual-timescale learning via evolution and consolidation,
- dynamic modularization without routing or paging.

This synthesis produces a computational substrate that is neither a statistical language model, nor a hypernetwork, nor a mixture-of-experts system, nor a recurrent architecture. Instead, it constitutes a distinct class of deterministic procedural cognitive architectures, characterized by stable structure, bounded analog variation, and interpretable modular dynamics.

D.6 Summary

Procedural DMLG represents a conceptual departure from existing machine learning paradigms. Its combination of deterministic traversal, procedural micro-model generation, fixed geometry, and canonical graph semantics yields a unique architecture that aligns more closely with biological cognition than with classical neural models. This distinction underscores its value as a foundation for future research into modular, interpretable, and cognitively meaningful artificial systems.

E Triadic Cosmos Ecosystem Context

The present paper is part of the broader *Triadic Cosmos* ecosystem: a unified research program that develops a compositional ontology of intelligence together with modular, inspectable computational architectures. The ecosystem spans conceptual theory, executable reference implementations, and fully documented empirical demonstrations, forming a coherent framework for studying hybrid intelligent systems.

All materials, extended notes, and evolving documentation are publicly available in the open GitHub project:

`https://github.com/triadic-cosmos`

Within the `triadic-system` repository, a complete reference implementation of the DMLG and GLP engines, the narrator pipeline, and all supporting modules is provided under the AGPL-3.0 license.

References

- [1] J. De Witte, “GLP–Graph Behaviour Under Hyperparameter Variation,” Triadic Cosmos Ecosystem, 2026.
- [2] J. De Witte, “The GLP–Graph Architecture: Combining GLP–Mono with DMLG,” Triadic Cosmos Ecosystem, 2026.
- [3] J. De Witte, “The GLP–Mono Architecture,” Triadic Cosmos Ecosystem, 2026.
- [4] J. De Witte, “A Narrator Pipeline driven by DMLG: Structural Hybrid Intelligence,” Triadic Cosmos Ecosystem, 2026.
- [5] J. De Witte, “Language as Graph Traversal: The DMLG Architecture,” Triadic Cosmos Ecosystem, 2026.